

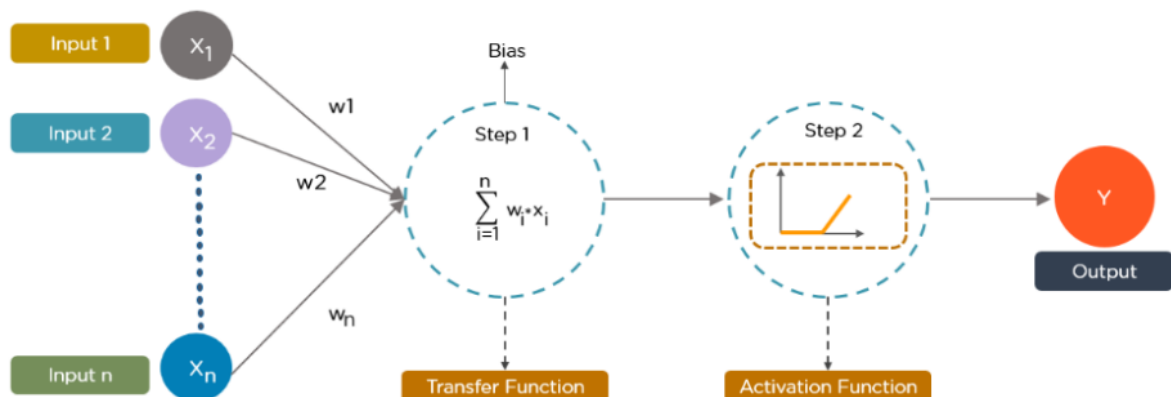
Binary and Multiclass Classification using Artificial Neural Networks (ANNs)

Aim:

Build an Artificial Neural Network to implement binary classification task using the Back-Propagation algorithm and test the same using appropriate datasets.

Artificial Neural Network (ANN):

Artificial Intelligence Networks (ANN) are computational models inspired by the human brain's neural structure. They consist of neurons (or nodes) organized in layers: the input layer, one or more hidden layers, and the output layer.



Activation Function:

1. **Rectified Linear Unit (ReLU)**
 - Used in hidden layers to add non-linearity
2. **Sigmoid**
 - Used in output layer for classification

Part A - Binary Classification:

Binary Classification is a supervised learning task where the model predicts one of the two possible classes (e.g., 0 or 1, True or False). The output layer of the ANN in binary classification typically uses the **sigmoid activation function** which outputs a probability between 0 and 1. For binary classification:

- Output layer has 1 neuron.
- Activation function: Sigmoid.
- Loss function: Binary Cross entropy.

Sigmoid Function:

The sigmoid activation function maps values between 0 and 1 and is defined as:

$$s(z) = 1 / (1 + e^{-z})$$

It is suitable for probability estimation in binary problems.

Binary Crossentropy Loss:

This loss function measures the performance of a classification model whose output is a probability value between 0 and 1.

Experiment: Implement ANN for Binary Classification Task:

❖ Dataset:

PIMA Indian Diabetes Dataset

To download the dataset, visit <https://www.kaggle.com/>. Search for the required dataset and download it.

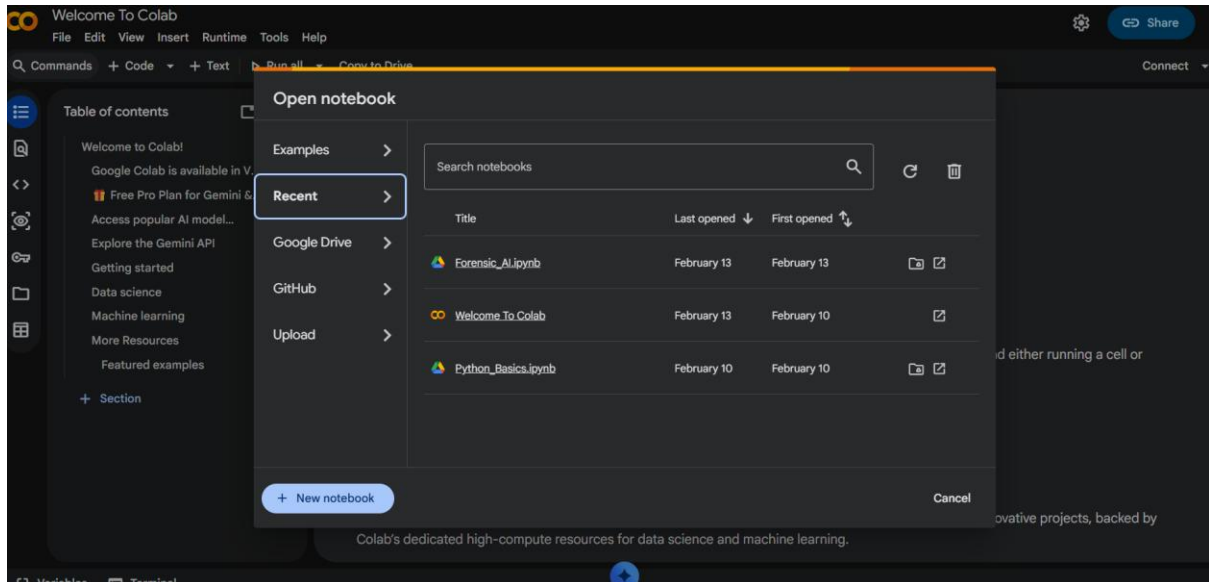
❖ Requirements:

- Python 3.0 and above
- Anaconda Jupyter or Google Colab
- Pandas, numpy, seaborn, matplotlib, scikit-learn

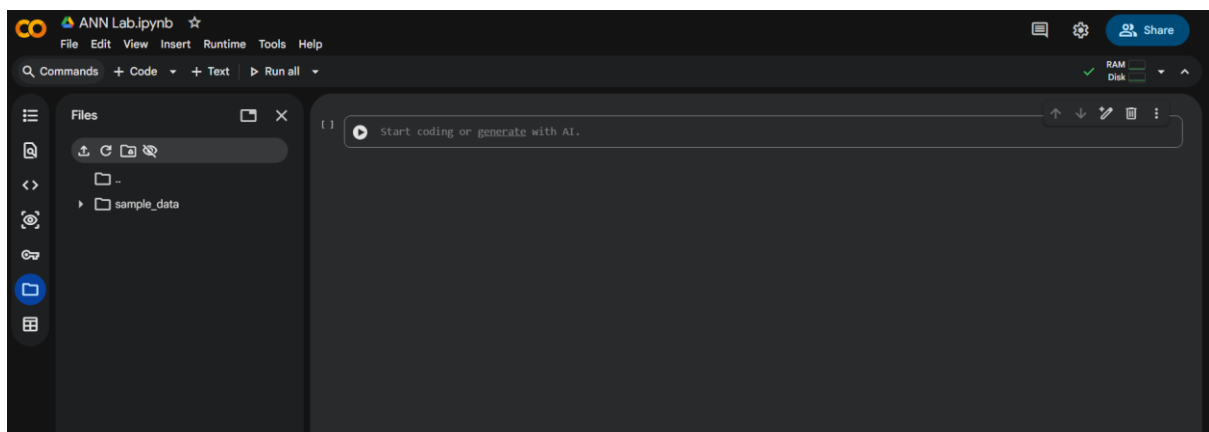
❖ Opening IDE:

First step is to open an IDE where we will be writing out code and analysing results. Google Colab is a hosted Jupyter Notebook service that requires no setup to use and provides the free access to computing resources, including GPUs and TPUs.

On your google search bar, write Google Colab and select on the first link. The following window will appear.

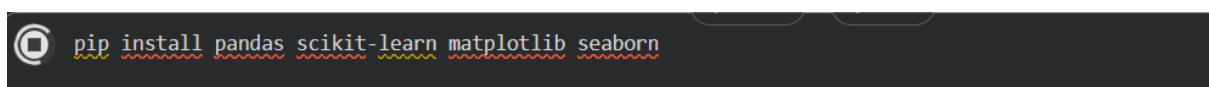


Click on “New Notebook”. A new notebook will be created. You have to rename your notebook with “Your_name-Lab01”. You will see the following window.



❖ Install the required Libraries:

Following libraries need to be downloaded.



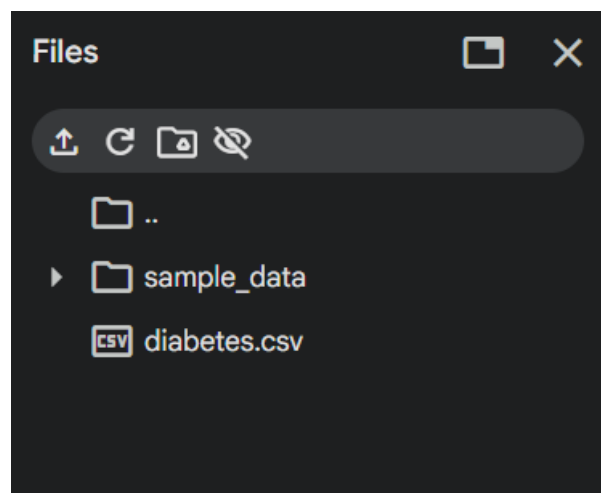
❖ Reading CSV File:

The dataset is stored in csv format. CSV refers to comma separated values which means that the data is available in tabular format where rows represent samples and columns represent features. Some sample records of dataset are shown below:

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
6	148	72	35	0	33.6	0.627	50	1
1	85	66	29	0	26.6	0.351	31	0
8	183	64	0	0	23.3	0.672	32	1
1	89	66	23	94	28.1	0.167	21	0
0	137	40	35	168	43.1	2.288	33	1
5	116	74	0	0	25.6	0.201	30	0
3	78	50	32	88	31	0.248	26	1
10	115	0	0	0	35.3	0.134	29	0
2	197	70	45	543	30.5	0.158	53	1

Before building ANN, we need to first load the data into Colab, read and analyze it.

Right click on the opened window and upload your dataset file here. You will now be able to see your csv file under the files tab.



Now we will read the dataset. To read any tabular data, we use pandas library.

```
import pandas as pd

data = pd.read_csv("diabetes.csv")
```

Now we will display first five rows of the dataset.

data.head()

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

In the same way, we can also display the last five rows of the dataset.

data.tail()

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
763	10	101	76	48	180	32.9	0.171	63	0
764	2	122	70	27	0	36.8	0.340	27	0
765	5	121	72	23	112	26.2	0.245	30	0
766	1	126	60	0	0	30.1	0.349	47	1
767	1	93	70	31	0	30.4	0.315	23	0

Now we will display some basic information about the dataset.

data.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Pregnancies            768 non-null    int64
1   Glucose                768 non-null    int64
2   BloodPressure          768 non-null    int64
3   SkinThickness          768 non-null    int64
4   Insulin                768 non-null    int64
5   BMI                    768 non-null    float64
6   DiabetesPedigreeFunction 768 non-null    float64
7   Age                    768 non-null    int64
8   Outcome                768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB

```

Some other information about the dataset features.

data.describe()										
...	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome	
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958	
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951	
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000	
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000	
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000	
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000	
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000	

Task Questions:

- Print number of rows and columns.
- Identify target column.
- Check if any missing values exist.

❖ Feature and Target Separation:

In supervised learning, input features map to X and output maps to labels (Y).

- $X \rightarrow$ Input features
- $y \rightarrow$ Target labels

ANN learns mapping:

$$f(X) \rightarrow y$$

If we don't separate, model won't know what to predict.

```
x = data.drop("Outcome", axis=1)
y = data["Outcome"]
```

Task Questions:

- Display first five examples of x and y.
- Print shape of x and y.
- What happens when we include target into x?
- Explain what axis =1 means?

❖ Train-Test Split:

Why do we need to split data?

If we train and test on same data:

- Model memorizes

- No real evaluation

Split ensures:

- Model generalization
- Unseen data testing

Common practice:

- 80% training
- 20% testing

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    x, y,
    test_size=0.2,
    random_state=42
)
```

Task Questions:

- Change test_size to 0.3.
- What happens if random_state is removed?
- Print size of train and test sets.

❖ Feature Scaling:

To implement ANN classifier, all the feature values should have same range. Or else, the features with higher values will dominate the decision boundary. To avoid this, we will first scale the features to bring them to a particular range.

Example:

- Age → 1–100
- Glucose → 0–200
- BMI → 0–50

Large values dominate gradient updates.

StandardScaler transforms data:

$$z = \frac{x - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Task Questions:

- Print first 5 rows before and after scaling.
- What happens if we scale entire dataset before splitting?
- Why do we use fit_transform() only on training data?

❖ Understanding ANN (MLP Classifier):

ANN consists of:

- Input layer
- Hidden layer(s)
- Output layer

Each neuron computes:

$$z = \sum w_i x_i + b$$

MLPClassifier performs:

- Forward propagation
- Backpropagation
- Gradient descent optimization

Key hyperparameters:

- hidden_layer_sizes
- activation
- solver
- max_iter


```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(
    hidden_layer_sizes=(16, 8),
    activation='relu',
    solver='adam',
    max_iter=300,
    random_state=42
)

model.fit(X_train, y_train)
```

Task Questions:

- Change hidden layers to (32,16).
- Change activation to 'tanh'.
- Increase max_iter to 500.

❖ Prediction and Evaluation:

After training:

Model predicts labels for unseen data.

Evaluation metrics:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Task Questions:

- Interpret confusion matrix.
- Identify false positives.
- Is accuracy enough for medical dataset?

- Which metric is most important here?

❖ **Plot Training Loss:**

Loss function measures error during training.

Goal:

Minimize loss over iterations.

If loss:

- Decreases → learning properly
- Oscillates → learning rate problem
- Increases → model unstable

```
import matplotlib.pyplot as plt

plt.plot(model.loss_curve_)
plt.title("Training Loss Curve")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.show()
```

Task Questions:

- What happens if max_iter is small?
- Does loss always decrease smoothly?

Part B-Multiclass Classification:

Multiclass classification is used when the output has more than two classes (e.g., 0,1,2).

For multiclass classification:

- Output layer has number of neurons equal to number of classes.
- Activation function: **Softmax**.
- Loss function: **Sparse Categorical Crossentropy** (for integer labels) or **Categorical Crossentropy**(for one-hot labels).

Softmax Function:

Softmax converts logits into probabilities such that the sum of probabilities equals 1.

```

data = pd.read_csv("Iris.csv")

X = data.drop("Species", axis=1)
y = data["Species"]

from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
y = encoder.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = MLPClassifier(hidden_layer_sizes=(20,10), max_iter=300)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

```

Lab Exercise:

- Perform Binary Classification on Breast Cancer Wisconsin (Diagnostic) Data Set. Download link: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>
- Perform Binary Classification on Heart Disease UCI Dataset. Download Link: <https://www.kaggle.com/code/mragpavank/heart-disease-uci/input>
- Perform Multi-Class Classification on Wine Quality Scores Dataset. Download Link: <https://www.kaggle.com/datasets/yasserh/wine-quality-dataset>
- Perform Multi-Class Classification on below dataset. Download link: <https://www.kaggle.com/code/balyan2521/vitamin-deficiency-disease-prediction-system/input>

Submission Instructions:

Prepare a PDF document that should contain the answers to all your Lab Questions. For Lab exercise Questions, add your code and output snapshots. Your lab report should be in the form of a complete document including cover page, proper formatting and required output screenshots. Name your file with your name_roll_no_Labx. Where x refers to Lab number (1,2,3....).